

Os skill 17

17(a) List Active Jobs

What it does: Lists active background jobs by displaying their job ID, process ID, and current status.

```
GNU nano 8.7.1 skill17a.c *
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

typedef struct
{
    int job_id;
    pid_t pid;
    char state[20];
} Job;

int main()
{
    Job jobs[2];
    pid_t p1, p2;

    p1 = fork();

    if (p1 == 0)
    {
        sleep(5);
        return 0;
    }

    p2 = fork();

    if (p2 == 0)
    {
        sleep(5);
        return 0;
    }

    jobs[0].job_id = 1;
    jobs[0].pid = p1;
    snprintf(jobs[0].state, sizeof(jobs[0].state), "RU

    jobs[1].job_id = 2;
```

```

jobs[0].job_id = 1;
jobs[0].pid = p1;
snprintf(jobs[0].state, sizeof(jobs[0].state), "RUNNING");

jobs[1].job_id = 2;
jobs[1].pid = p2;
snprintf(jobs[1].state, sizeof(jobs[1].state), "RUNNING");

printf("\nActive Jobs:\n");

printf("[%d] PID: %d  Status: %s\n",
       jobs[0].job_id,
       jobs[0].pid,
       jobs[0].state);

printf("[%d] PID: %d  Status: %s\n",
       jobs[1].job_id,
       jobs[1].pid,
       jobs[1].state);

printf("\nJob listing completed.\n");

waitpid(p1, NULL, 0);
waitpid(p2, NULL, 0);

return 0;
}

```

```

sri_dhruthi@Dhruthi:~$ nano skill17a.c
sri_dhruthi@Dhruthi:~$ gcc skill17b.c -o skill17b
cc1: fatal error: skill17b.c: No such file or directory
compilation terminated.
sri_dhruthi@Dhruthi:~$ gcc skill17a.c -o skill17a
sri_dhruthi@Dhruthi:~$ ./skill17a

Active Jobs:
[1] PID: 964  Status: RUNNING
[2] PID: 965  Status: RUNNING

Job listing completed.

```

17(b) Foreground Job Switching

What it does: Identifies a target job, transfers execution to it, waits for completion, and then returns control to the shell.

```
sri_dhruthi@Dhruthi: ~  
GNU nano 8.7.1 skill17b.c *  
#include <stdio.h>  
#include <unistd.h>  
#include <sys/wait.h>  
  
int main()  
{  
    pid_t pid;  
    int status;  
  
    pid = fork();  
  
    if (pid < 0)  
    {  
        perror("fork");  
        return 1;  
    }  
  
    if (pid == 0)  
    {  
        printf("Foreground job started.\n");  
        printf("PID: %d\n", getpid());  
  
        sleep(3);  
  
        printf("Foreground job completed.\n");  
  
        return 0;  
    }  
  
    printf("Shell: Target job identified.\n");  
    printf("Shell: Transferring control to PID %d\n", pid);  
  
    waitpid(pid, &status, 0);  
  
    if (WIFEXITED(status))  
    {  
        printf("Shell: Job completed successfully.\n");  
    }  
}
```

```
sri_dhruthi@Dhruthi:~$ nano skill17b.c  
sri_dhruthi@Dhruthi:~$ gcc skill17b.c -o skill17b  
sri_dhruthi@Dhruthi:~$ ./skill17b  
Shell: Target job identified.  
Shell: Transferring control to PID 1003  
Foreground job started.  
PID: 1003
```